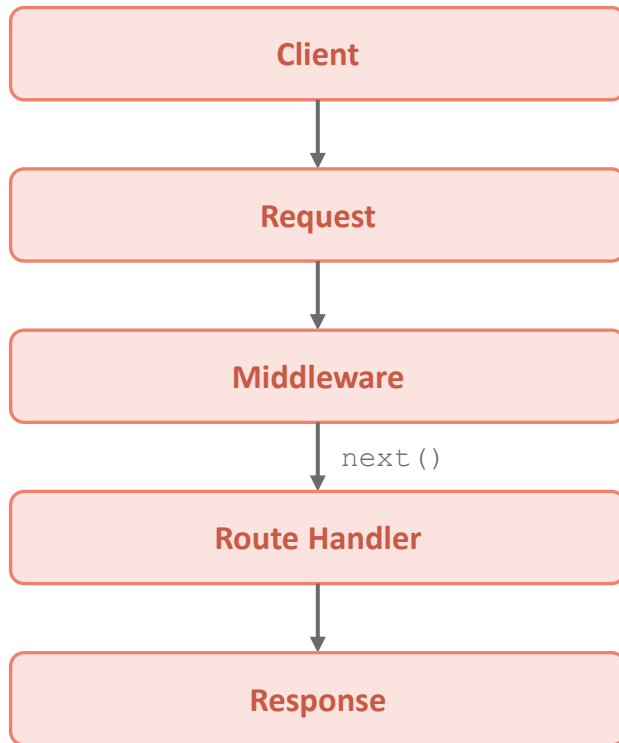


Node.js / Express Middleware

- **Middleware** is a function that runs between the incoming request and the final response.
- Every middleware function has access to the request (req), response (res), and a **next()** function.
- Calling **next()** hands control to the next middleware in the chain — without it, the request hangs.

```
app.use((req, res, next) => {  
  console.log("Middleware executed");  
  next();  
});
```



1. Application-Level Middleware

- Registered with `app.use()` and runs for **every incoming request**, regardless of route.
- Commonly used for logging, request timing, CORS, or parsing — anything that should apply globally.
- Order matters: middleware registered first runs first.

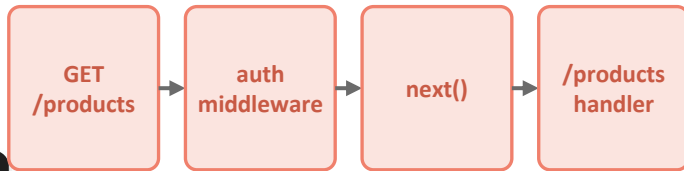


```
app.use((req, res, next) => {  
  console.log(req.method, req.url);  
  next();  
});
```

2. Route-Specific Middleware

- Runs only for a particular route — passed as an argument between the path and the final handler.
- Useful for logic that only some routes need, like authentication or input validation.
- Multiple middleware functions can be chained for one route.

Route middleware flow



```
const auth = (req, res, next) => {  
  console.log("Authentication");  
  next();  
};  
app.get("/products", auth, (req, res) => {  
  res.json({ message: "Products" });  
});
```

3. Built-in Middleware

- Express ships with middleware for common tasks, e.g. **express.json()** and **express.urlencoded()**.
- **express.json()** parses incoming requests with a JSON body:
- ...and makes the parsed data available through **req.body** in your route handlers.

```
{  
  "name": "Laptop",  
  "price": 50000  
}
```

Available as: **req.body**



```
app.use(express.json());  
app.use(  
  express.urlencoded({ extended: true })  
);
```

4. Third-Party Middleware

- Published as npm packages and installed separately — they extend Express beyond its built-ins.
- Common examples:
 - **cors** — enable cross-origin requests
 - **morgan** — HTTP request logging
 - **helmet** — sets security-related HTTP headers
 - **cookie-parser** — parses cookies into req.cookies



```
import cors from "cors";  
app.use(cors());
```

5. Error-Handling Middleware

- Defined with **four parameters**: (err, req, res, next).
- The first parameter, **err**, is what tells Express this is an error handler rather than regular middleware.
- Express automatically routes errors here when next(err) is called, or when a handler throws.



```
app.use((err, req, res, next) => {  
  console.log(err);  
  res.status(500).json({  
    message: "Something went wrong"  
  });  
});
```

6. 404 Middleware

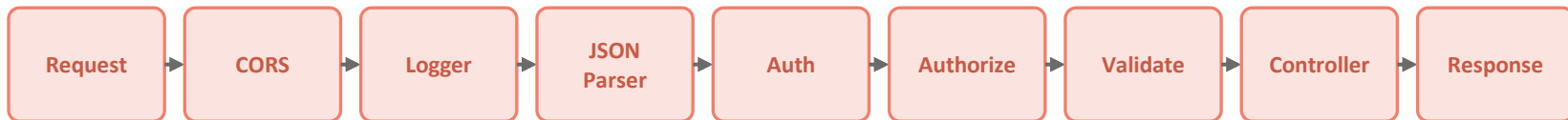
- Catches any request that didn't match a defined route — Express falls through to it when nothing else handled the request.
- Must be placed after all other routes and middleware, or it will intercept requests meant for real routes.



```
app.use((req, res) => {  
  res.status(404).json({  
    message: "Page not found"  
  });  
});
```

Real-World Middleware Chain

A production API typically chains several layers of middleware before reaching the controller:



```
app.use(cors());
app.use(express.json());
app.use(logger);
app.get(
  "/products",
  authenticate, authorize, validate,
  getProducts
);
```


Why Is next() Important?

- **next()** passes control to the next middleware or route handler in the chain.
- If a middleware never calls next() (and never sends a response), the request hangs forever.
- Calling next(err) skips straight to the error-handling middleware.



```
const logger = (req, res, next) => {  
  console.log("Request received");  
  next();  
};
```